

Latency-Centric RAN Orchestration with World Model for Satellite-Assisted Mobile Edge Computing

Dong-eui Kim^{*†}, Seonghoon Kim[†], and Jihwan P. Choi[†]

^{*}Space Exploration Engineering Program, [†]Dept. of Aerospace Engineering, KAIST, Korea
{argee_im, seonghoon, jhch}@kaist.ac.kr

Abstract—As the sixth-generation (6G) network evolves to include low Earth orbit (LEO) satellites, it is envisioned to use radio access network (RAN) orchestration for artificial intelligence (AI) task offloading in LEO satellite-assisted mobile edge computing (SAMEC), where satellites serve as relays to forward tasks to ground-based edge servers. However, the realization of efficient orchestration faces fundamental challenges: (i) heterogeneous latency requirements across diverse services, (ii) partial observability stemming from both stale state information caused by propagation delays and low observation resolution, and (iii) stochastic uncertainty such as task arrivals and channel fluctuations. This paper formulates the resource management and load balancing problem as a constrained partially observable Markov decision process (CPOMDP) and proposes a novel hierarchical framework that learns stochastic environmental dynamics via a world model. By combining planning-based imagination for sample-efficient long-term optimization with heuristic scheduling for short-term optimization, our approach maintains latency constraint deviations below 20 ms, significantly outperforming model-free baselines.

Index Terms—satellite-assisted, MEC, world model, CPOMDP.

I. INTRODUCTION

The explosive growth of artificial intelligence (AI) applications in the sixth-generation (6G) era demands edge computing (EC) capabilities to offload computationally intensive and delay-sensitive workloads from resource-constrained user equipments (UEs). While terrestrial networks are densely deployed to support such services, they struggle with scalability and coverage gaps in remote areas. As a promising solution, non-terrestrial networks (NTN), specifically low Earth orbit (LEO) satellites, are emerging to provide seamless global coverage. This integration of AI and EC within NTN prompts a paradigm shift toward communication and computing integrated networks (CCIN) [1], enabling joint resource management to guarantee stringent quality of service (QoS) requirements.

Unlike EC in terrestrial radio access networks (RAN) [2], LEO satellites are inherently limited by size, weight, and power (SWaP) constraints, particularly in terms of onboard energy and computing capacity. Although satellite-borne edge computing architectures have been explored [3], practical SWaP limitations necessitate a satellite-assisted architecture where satellites primarily function as access and relay nodes, while computationally intensive AI tasks are offloaded to ground-based

computing platforms. While a centralized ground-based controller can manage the dynamic topology and logical mapping between LEO satellites and terrestrial infrastructure [4], the task of effectively allocating resources and coordinating dynamic load balancing across heterogeneous platforms faces three fundamental challenges: (i) heterogeneous latency requirements across diverse services, (ii) partial observability stemming from both stale state information caused by propagation delays in signaling between the ground controller and the serving LEO satellite, and low observation resolution resulting from discrete snapshot periods that fail to capture the fast-fading dynamics of the environment, and (iii) stochastic uncertainty in UE mobilities, AI task arrivals, and channel fluctuations.

Despite the significance of the aforementioned challenges, it is difficult to find research work that explicitly addresses (i), (ii), and (iii) simultaneously in LEO satellite networks with mobile edge computing (MEC) capability. Prior stochastic optimization work based on Lyapunov optimization [5] has addressed (iii). However, this line of work often relies on idealized assumptions such as homogeneous latency constraints and perfect channel state information (CSI), thereby neglecting (i) and (ii). To bridge this gap, the limited existing studies have primarily utilized model-free deep reinforcement learning (DRL) [6] or recurrent neural network (RNN)-based DRL [7]. These reactive methods fail to internalize environmental dynamics, leading to poor sample efficiency and credit assignment problems under partial observability. To overcome such issues, we propose a novel hierarchical framework based on *world models* that acquires situational awareness by internalizing causal dynamics. Instead of reacting to raw inputs, our model compresses high-dimensional observations into compact latent states, effectively filtering out noisy data to construct a robust internal representation. This capability enables the agent to imagine future trajectories and proactively manage RAN operations, satisfying strict latency requirements with high sample efficiency. Crucially, the proposed agent maintains inference complexity comparable to model-free baselines, facilitating real-time orchestration in LEO satellite-assisted MEC (SAMEC) systems. The main contributions are as follows.

- **Hierarchical Framework for Constrained Partially Observable Markov Decision Process (CPOMDP):** To address (i), we formulate the energy-efficient resource management problem as a CPOMDP. We propose a hierarchical framework that solves the Lagrange dual

This work was supported in part by the KASA, Korea AeroSpace Administration, under Grant 2022M1A3C2069728, Future Space Education Center, and in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2025-24523397).

problem, maintaining latency violations below 20 ms, which significantly outperforms model-free baselines in terms of constraint satisfaction.

- **Sample-Efficient Dynamics Learning via World Model:** To overcome (ii) and (iii), we employ a world model based on evidence lower bound (ELBO) [8] and recurrent state-space model (RSSM) [9] to extract robust latent representations. By utilizing planning-based imagination, our approach extends the utility of training data over the planning horizon. This enables substantially faster convergence and higher sample efficiency, providing the agent with the situational awareness necessary for proactive orchestration in dynamic environments.

The rest of this paper is organized as follows: Section II provides the system model. Section III formulates an optimization problem and proposes algorithms. Sections IV and V present numerical results and conclusion, respectively.

II. SYSTEM MODEL

Our system operations are modeled in a hierarchical discrete-time domain, where time is divided into frames of duration ΔT , indexed by an integer $t \geq 0$. Each frame consists of S equal time slots with local index $s \in \mathcal{S} = \{1, \dots, S\}$. The global time slot index is given by $t_s = t \cdot S + s$. Each network platform performs scheduling decisions at t , while individual UE resource allocations occur at t_s . Our system aims to accommodate a variety of AI workloads generated by the UEs through network orchestration. The UEs are indexed by $u \in \mathcal{U} = \{1, \dots, U\}$, where U is the number of UEs. Each UE is classified into either latency-sensitive services (LSSs) $\mathcal{U}^{\text{LSS}}$ requiring low latency and small data volumes, or latency-tolerant services (LTSSs) $\mathcal{U}^{\text{LTS}}$ with more relaxed QoS requirements and larger data volumes. Each UE belongs to only one class, i.e., $\mathcal{U}^{\text{LSS}} \cap \mathcal{U}^{\text{LTS}} = \emptyset$. We use square brackets $[\cdot]$ for time-indexed system variables and parentheses $(\cdot)$ for performance metrics derived per packet.

We consider a LEO SAMEC system, illustrated in Fig. 1, which is coordinated by a centralized terrestrial orchestrator and comprises multiple terrestrial base stations (TBSs) in urban areas, each co-located with a computing platform (CPP) denoted as $i \in \mathcal{I} = \{1, \dots, I\}$ with the number of TBSs I , and a single LEO satellite denoted as $i = 0$. We assign each UE to a communication platform (CMP) based on Euclidean distance $D_{u,i}[t]$ from UE u to TBS i at t . Each UE connects to the nearest TBS within the maximum coverage radius R_{th} , i.e., the CMP index $i_u[t] = \text{argmin}_{i \in \mathcal{I}: D_{u,i}[t] \leq R_{\text{th}}} D_{u,i}[t]$ associated with u . If no TBS is within the range, LEO ($i_u[t] = 0$) serves the UE in remote areas outside of TBS coverage. Considering the computationally intensive nature of the target AI tasks, we assume that the LEO satellite forwards the tasks to a selected terrestrial CPP. This creates two link types: the uplink (UL) from UE to its associated CMP, denoted by UL, and the relay link (ReL) from LEO to a selected TBS, denoted by ReL. We assume that the processed data is much smaller than the original task size, and thus omit a return link of the tasks.

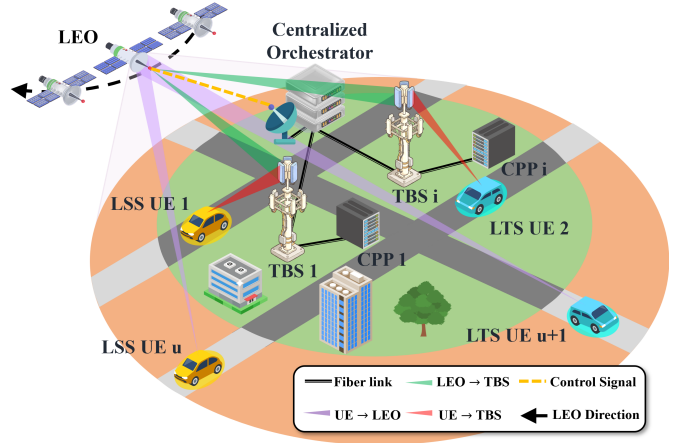


Figure 1. The LEO SAMEC system model featuring a LEO satellite, multiple terrestrial base stations (TBSs) that function as communication platforms (CMPs), terrestrial computing platforms (CPPs), latency-sensitive service (LSS) user equipments (UEs), and latency-tolerant service (LTS) UEs.

Within the multiple TBSs available for relay, we employ a probabilistic TBS selection policy to balance workload from the remote area across the CPPs. For each service type $\text{sl} \in \mathcal{S} = \{\text{LSS}, \text{LTS}\}$, a probability $O_i^{\text{sl}}[t]$ represents the likelihood that a task from sl arriving at t will be forwarded to TBS i for computing. This forms a probability mass function (PMF) over all available TBSs, which satisfies:

$$0 \leq O_i^{\text{sl}}[t] \leq 1, \quad \forall i \in \mathcal{I}, \text{sl} \in \mathcal{S}, \quad (1)$$

$$\sum_{i \in \mathcal{I}} O_i^{\text{sl}}[t] = 1, \quad \forall \text{sl} \in \mathcal{S}. \quad (2)$$

Based on this policy, a TBS is randomly selected according to PMF $\{O_i^{\text{sl}}[t]\}_{i \in \mathcal{I}}$ for each sl . We define $x_i^{\text{sl}}[t]$ as a binary indicator variable of TBS selection for remote area tasks, where $x_i^{\text{sl}}[t]$ equals 1 if CMP i is sampled from the PMF, and 0 otherwise. According to the selection, we define the UE sets associated with CMP i at t as $\mathcal{U}_i^{\text{UL,sl}}[t] = \{u \in \mathcal{U}^{\text{sl}} : i_u[t] = i \in \mathcal{I} \cup \{0\}\}$, and processed at CPP i as $\mathcal{U}_i^{\text{CPP,sl}}[t] = \mathcal{U}_i^{\text{UL,sl}}[t] \cup \{u \in \mathcal{U}_0^{\text{UL,sl}}[t] \mid x_i^{\text{sl}}[t] = 1\}$, which include both direct and LEO-relayed UEs at TBS i for sl .

A. Communication Model

We assume that all CMPs employ orthogonal multiple access (OMA) to serve UEs, where each link utilizes an isolated radio resource to avoid interference. Given transmit power P^{TX} , antenna gains $G^{\text{TX/RX}}$, channel $h[t_s]$, noise power spectral density N_0 , and bandwidth $B[t_s]$, the signal-to-noise ratio (SNR) is modeled as $\text{SNR}[t_s] = \frac{P^{\text{TX}} \cdot G_u^{\text{TX}} \cdot G_i^{\text{RX}} \cdot |h[t_s]|^2}{N_0 B[t_s]}$ at t_s . The instantaneous channel $h[t_s]$ is modeled as $h[t_s] = \sqrt{\frac{\kappa[t] \Gamma[t]}{\kappa[t] + 1}} + \sqrt{\frac{\Gamma[t]}{\kappa[t] + 1}} \alpha[t_s]$, with $\alpha[t_s] \sim \mathcal{CN}(0, 1)$ representing small-scale fading, $\kappa[t]$ as the Rician K-factor, and $\Gamma[t]$ as the large-scale fading coefficient. For each link type $\ell \in \{\text{UL}, \text{ReL}\}$, we instantiate the SNR as $\text{SNR}_{u,i}^{\ell}[t_s]$. For the UL, u transmits to CMP i with the transmit power P_u^{TX} , UE antenna gain G_u^{TX} , CMP i receiving antenna gain G_i^{RX} , channel

$h_{u,i}^{\text{UL}}[t_s]$, and allocated bandwidth $B_{u,i}^{\text{UL}}[t_s]$. For the ReL, the LEO forwards u 's packet to TBS i^* with LEO transmit power P_0^{TX} , LEO antenna gain G_0^{TX} , receiving antenna gain $G_{i^*}^{\text{RX}}$ of TBS i^* , channel $h_{u,i^*}^{\text{ReL}}[t_s]$, and allocated bandwidth $B_{u,i^*}^{\text{ReL}}[t_s]$. Given the total bandwidth $B_i^{\text{max},\ell}$ for CMP i and ℓ , let $B_i^\ell[t]$ denote the allocated bandwidth at t , satisfying

$$0 \leq B_i^{\text{UL}}[t] \leq B_i^{\text{max},\text{UL}}, \quad \forall i \in I \cup \{0\}, t, \quad (3)$$

$$0 \leq B_i^{\text{ReL}}[t] \leq B_i^{\text{max},\text{ReL}}, \quad \forall i \in I, t. \quad (4)$$

The service-level bandwidth allocation $B_i^{\ell,\text{sl}}[t]$ for each $\text{sl} \in \mathcal{SL}$ satisfies

$$\sum_{\text{sl}} B_i^{\text{UL},\text{sl}}[t] \leq B_i^{\text{UL}}[t], \quad \forall i \in I \cup \{0\}, t, \quad (5)$$

$$\sum_{\text{sl}} B_i^{\text{ReL},\text{sl}}[t] \leq B_i^{\text{ReL}}[t], \quad \forall i \in I, t. \quad (6)$$

For UL transmission, $b_{u,i}^{\text{UL}}[t]$ is the UL bandwidth fraction for u on CMP i at t_s , where $B_{u,i}^\ell = b_{u,i}^\ell B_i^\ell$, satisfying

$$\sum_{u \in \mathcal{U}_i^{\text{UL},\text{sl}}[t]} b_{u,i}^{\text{UL}}[t_s] \leq 1, \quad \forall i \in I \cup \{0\}, \text{sl}, t, s. \quad (7)$$

For each ReL, $b_{u,i}^{\text{ReL}}[t]$ is the proportion of relay bandwidth allocated to u within the set of UEs employing ReL for computing offload, $\mathcal{U}_i^{\text{ReL},\text{sl}}[t] = \mathcal{U}_i^{\text{CPP}}[t] \setminus \mathcal{U}_i^{\text{UL},\text{sl}}[t]$, satisfying

$$\sum_{u \in \mathcal{U}_i^{\text{ReL},\text{sl}}[t]} b_{u,i}^{\text{ReL}}[t_s] \leq 1, \quad \forall i \in I, \text{sl}, t, s. \quad (8)$$

For both link types, the bandwidth fraction is bounded by

$$0 \leq b_{u,i}^\ell[t_s] \leq 1, \quad \forall u, i, \ell, t, s. \quad (9)$$

For any t_s , we define an instantaneous achievable rate as $r_{u,i}^\ell[t_s] = b_{u,i}^\ell[t_s] \cdot B_i^{\ell,\text{sl}}[t] \cdot \log_2 \left(1 + \text{SNR}_{u,i}^\ell[t_s] \right)$.

B. Computing Model

Each CPP's data processing is modeled to reflect the unique characteristics of AI workloads, which are often constrained by both throughput and memory bandwidth. We assume that each CPP is equipped with a graphics processing unit (GPU) that supports partitioning into multiple resource-isolated instances. Given the maximum computing throughput C_i^{max} and memory bandwidth F_i^{max} for CPP $i \in I$, let $C_i[t]$ and $F_i[t]$ denote the total available computing throughput [FLOP/s] and memory bandwidth [byte/s] at t , respectively, which satisfy:

$$0 \leq C_i[t] \leq C_i^{\text{max}}, \quad 0 \leq F_i[t] \leq F_i^{\text{max}}, \quad \forall i \in I, t. \quad (10)$$

We assume the GPU's partitioning system couples computing and memory resources; therefore, we use a single control variable $\rho_i^{\text{sl}}[t] \in [0, 1]$ to jointly allocate both resources to each sl. The resource share variable $\rho_i^{\text{sl}}[t]$ satisfies:

$$\sum_{\text{sl}} \rho_i^{\text{sl}}[t] \leq 1, \quad \forall i \in I, t, \quad (11)$$

and determines the allocated computing throughput $C_i^{\text{sl}}[t]$ and memory bandwidth $F_i^{\text{sl}}[t]$ for each sl, which are expressed as:

$$C_i^{\text{sl}}[t] = \rho_i^{\text{sl}}[t] C_i[t], \quad F_i^{\text{sl}}[t] = \rho_i^{\text{sl}}[t] F_i[t], \quad \forall i \in I, \text{sl}, t. \quad (12)$$

Within this instance, the fraction of processing time allocated to u on CPP i is denoted by $f_{u,i}^{\text{CPP}}[t_s]$, satisfying:

$$\sum_{u \in \mathcal{U}_i^{\text{CPP}}[t]} f_{u,i}^{\text{CPP}}[t_s] \leq 1, \quad \forall i \in I, \text{sl}, t, s, \quad (13)$$

$$0 \leq f_{u,i}^{\text{CPP}}[t_s] \leq 1, \quad \forall u, i \in I, \text{sl}, t, s. \quad (14)$$

The execution rate of u 's workload [bit/s] is modeled using a dual-bottleneck approach based on the Roofline model [10], which is $r_{u,i}^{\text{CPP}}[t_s] = \eta_{\text{CPP}}[t] \frac{\min(C_i^{\text{sl}}[t], I_{\text{WL}} \cdot F_i^{\text{sl}}[t])}{\Gamma_{\text{WL}}} \cdot f_{u,i}^{\text{CPP}}[t_s]$, where $\eta_{\text{CPP}}[t]$ denotes the efficiency factor accounting for effects such as thermal throttling and system overhead, and I_{WL} [FLOP/byte] and Γ_{WL} [FLOP/bit] are the workload arithmetic intensity and computational cost per bit, respectively.

C. Communication Performance Metrics

Based on the achievable data rates, we define the latency for each communication hop. For the UL latency, for the packet of size $d_u(t_s)$ generated at t_s by u associated with $i_u[t]$, the number of slots for the UE $\rightarrow$ TBS or LEO transmission is

$$N_u^{\text{UL}}(t_s) = \min \left\{ N : \sum_{k=0}^{N-1} r_{u,i_u[t_s]+k}^{\text{UL}}[t_s+k] \cdot \frac{\Delta T}{S} \geq d_u(t_s) \right\}. \quad (15)$$

Therefore, the UL latency is $T_u^{\text{UL}}(t_s) = N_u^{\text{UL}}(t_s) \cdot \frac{\Delta T}{S}$. Meanwhile, if the UE is associated with LEO ($i_u[t] = 0$), the packet must be forwarded from LEO to a TBS during the relay phase. When the UL transmission is completed, we define the start time slot of the relay phase as $t'_s = t_s + N_u^{\text{UL}}(t_s)$. The number of slots allocated for this relay transmission is

$$N_u^{\text{ReL}}(t_s) = \min \left\{ N : \sum_{k=0}^{N-1} r_{u,i^*}^{\text{ReL}}[t'_s+k] \cdot \frac{\Delta T}{S} \geq d_u(t_s) \right\}, \quad (16)$$

where i^* denotes the TBS selected according to $x_{i^*}^{\text{sl}}[t']$ at the relay start frame $t' = \lfloor t'_s/S \rfloor$, with $\lfloor x \rfloor$ representing the floor operation, and this selection persists until the relay transmission is complete. The relay latency $T_u^{\text{ReL}}(t_s)$ is computed similarly, using $N_u^{\text{ReL}}(t_s)$. Therefore, the total communication latency for u depends on association, using an indicator function $\mathbf{1}_{\{\cdot\}}$:

$$T_u^{\text{CMP}}(t_s) = T_u^{\text{UL}}(t_s) + \mathbf{1}_{\{i_u[t]=0\}} \cdot T_u^{\text{ReL}}(t_s). \quad (17)$$

The overall communication energy comprises two link types. We adopt a linear power model [11] where the transmit/receive power $P_{u,i}^{\text{TX/RX},\ell}[t_s]$ for each link ℓ is defined as:

$$P_{u,i}^{\text{TX/RX},\ell}[t_s] = P_{\text{static}}^{\text{TX/RX}} + P_{\text{cir}}^{\text{TX/RX}} \cdot b_{u,i}^\ell[t_s] \cdot B_i^{\ell,\text{sl}}[t], \quad (18)$$

where $P_{\text{static}}^{\text{TX/RX}}$ and $P_{\text{cir}}^{\text{TX/RX}}$ denote static and circuit power per unit bandwidth, respectively. The UL energy is given by

$$E_u^{\text{UL}}(t_s) = \sum_{\tau=t_s}^{t_s+N_u^{\text{UL}}(t_s)-1} \left(P_{u,i}^{\text{TX,UL}}[\tau] + P_{u,i}^{\text{RX,UL}}[\tau] \right) \frac{\Delta T}{S}. \quad (19)$$

For u with $i_u[t] = 0$, the relay energy $E_u^{\text{ReL}}(t_s)$ is computed similarly, using $P_{u,i^*}^{\text{TX,ReL}}[t_s]$, $P_{u,i^*}^{\text{RX,ReL}}[t_s]$, and $N_u^{\text{ReL}}(t_s)$. Thus, the total communication energy is represented as:

$$E_u^{\text{CMP}}(t_s) = E_u^{\text{UL}}(t_s) + \mathbf{1}_{\{i_u[t]=0\}} E_u^{\text{ReL}}(t_s). \quad (20)$$

D. Computing Performance Metrics

Similarly to Subsection II-C, computing latency for u $T_u^{\text{CPP}}(t_s)$ is derived from $N_u^{\text{CPP}}(t_s)$, which is calculated using $r_{u,i}^{\text{CPP}}[t]$. The computing power model [12] is expressed as:

$$P_{u,i}^{\text{CPP}}[t_s] = P_i^{\text{idle}} + P_i^{\text{sh}}[t] + P_{u,i}^{\text{dyn,sl}}[t_s]. \quad (21)$$

P_i^{idle} represents the idle power. The shared power $P_i^{\text{sh}}[t]$ distributes the GPU's base power P_i^{gpu} proportionally to instance performance variables C_i^{sl} and F_i^{sl} , as given by:

$$P_i^{\text{sh}}[t] = P_i^{\text{gpu}} \cdot \frac{w_{\text{sm}} C_i^{\text{sl}}[t] + w_{\text{mem}} F_i^{\text{sl}}[t]}{\sum_{\text{sl}' \in \mathcal{SL}} (w_{\text{sm}} C_i^{\text{sl}'}[t] + w_{\text{mem}} F_i^{\text{sl}'}[t])}. \quad (22)$$

The dynamic power $P_{u,i}^{\text{dyn,sl}}[t_s]$ is defined as:

$$P_{u,i}^{\text{dyn,sl}}[t_s] = r_{u,i}^{\text{CPP}}[t_s] \cdot \Gamma_{\text{WL}} \cdot \left(\varepsilon_i^{\text{C}} + \frac{\varepsilon_i^{\text{F}}}{I_{\text{WL}}} \right), \quad (23)$$

where ε_i^{C} and ε_i^{F} are the computing and memory energy coefficients, respectively. Finally, the total computing energy is denoted by:

$$E_u^{\text{CPP}}(t_s) = \sum_{\tau=t_s}^{t_s + N_u^{\text{CPP}}(t_s) - 1} P_{u,i}^{\text{CPP}}[\tau] \cdot \frac{\Delta T}{S}. \quad (24)$$

Therefore, the total energy consumption for u 's packet generated at t_s is given by $E_u^{\text{Total}}(t_s) = E_u^{\text{CMP}}(t_s) + E_u^{\text{CPP}}(t_s)$.

III. HIERARCHICAL OPTIMIZATION FRAMEWORK

We adopt a hierarchical optimization framework, as illustrated in Fig. 2, which decouples the energy minimization problem into two timescales. Joint optimization of platform-level decision-making and UE-level scheduling is computationally intractable due to two reasons. First, platform-level decisions must be made at time t but control all packets arriving throughout the future interval $[t, t+1)$, whose arrival times, channel states, and task characteristics are unknown at decision time, precluding deterministic optimization approaches. Addressing this requires learning a predictive model of the composite environment dynamics to anticipate system states under uncertainty. Second, even if future states were known, the UE-level scheduling problem itself would involve a high-dimensional decision space, making real-time optimal allocation difficult. Therefore, the inner scheduling policy must be designed with low computational complexity to ensure real-time feasibility.

A. Problem Formulation

Accordingly, we set an outer policy π_{O} to manage the variable set $\{B_j^{\ell}[t], C_i[t], F_i[t], B_j^{\ell,\text{sl}}[t], \rho_i^{\text{sl}}[t], \Theta_i^{\text{sl}}[t], O_i^{\text{sl}}[t]\}$, for all $\text{sl}, i \in \mathcal{I}$, and $j \in \mathcal{I} \cup \{0\}$, while the inner policy π_{I} performs UE-level scheduling via control variables $\{b_{u,i}^{\ell}[t_s], f_{u,i}^{\text{CPP}}[t_s]\}$. Variables managed by π_{O} , such as the relay selection probability $O_i^{\text{sl}}[t]$ and latency budget splitting variable $\Theta_i^{\text{sl}}[t]$, determined at t , must control all generated packets throughout $[t, t+1)$. The system objective is to minimize the long-term time-averaged

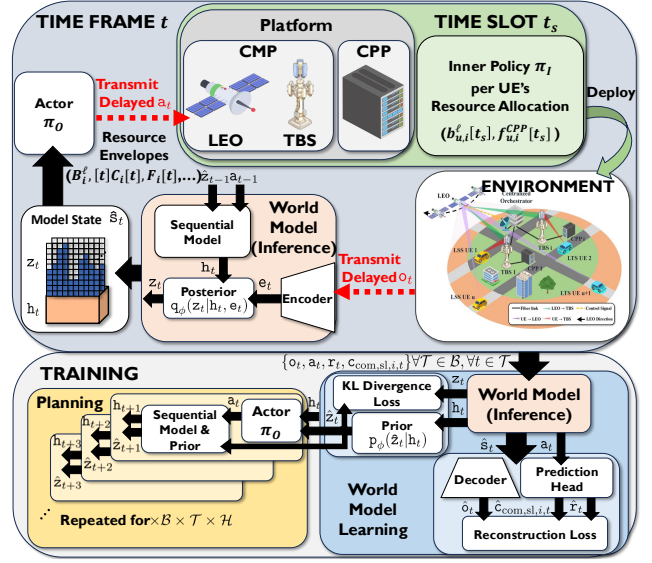


Figure 2. The proposed hierarchical optimization framework, illustrating the interaction between the world model-based outer policy at the time frame level and the heuristic inner policy at the time slot level.

total energy consumption, which includes both communication and computing energy across all UEs, given by:

$$J(\pi_{\text{O}}, \pi_{\text{I}}) = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \sum_{s=0}^{S-1} \sum_{u \in \mathcal{U}} E_u^{\text{Total}}(t_s). \quad (25)$$

We then formulate the energy minimization problem with per-packet latency constraints as follows:

$$\mathcal{P} : \min_{\pi_{\text{O}}, \pi_{\text{I}}} J(\pi_{\text{O}}, \pi_{\text{I}}) \quad (26a)$$

$$\text{s.t.} \quad (1) - (14)$$

$$T_u^{\text{CMP}}(t_s) \leq \Theta_{i_u[t]}^{\text{sl}}[t] T_u^{\text{max}}(t_s), \forall t, s, u \in \mathcal{U} \quad (26b)$$

$$T_u^{\text{CPP}}(t_s) \leq (1 - \Theta_{i_u[t]}^{\text{sl}}[t]) T_u^{\text{max}}(t_s), \forall t, s, u \in \mathcal{U} \quad (26c)$$

$$0 \leq \Theta_i^{\text{sl}}[t] \leq 1, \forall i \in \mathcal{I} \cup \{0\}, \text{sl} \in \mathcal{SL}, t, \quad (26d)$$

where $T_u^{\text{max}}(t_s)$ represents the maximum allowable latency for each packet generated at t_s , and $\Theta_{i_u[t]}^{\text{sl}}[t]$ splits $T_u^{\text{max}}(t_s)$ via constraints (26b) and (26c), enabling the system to minimize energy consumption while satisfying per-packet latency constraints. Here, the non-optimizable propagation delay is treated as a worst-case constant and deducted from the latency budget T_u^{max} . Therefore, optimization is focused on controllable queuing and processing delays. To pass optimization variables to the DRL agent, we define normalized utilization weights that map each resource to the $[0, 1]$ interval: $W_i^{\ell,B}[t] = \frac{B_i^{\ell}[t]}{B_i^{\text{max},\ell}}$ for all $i \in \mathcal{I} \cup \{0\}$ for communication bandwidth, $W_i^{\text{CPP},F}[t] = \frac{C_i[t]}{C_i^{\text{max}}}$ and $W_i^{\text{CPP},F}[t] = \frac{F_i[t]}{F_i^{\text{max}}}$ for all $i \in \mathcal{I}$ for computing throughput and memory bandwidth, respectively. For sl , allocation weights $W_i^{\ell,B,\text{sl}}[t] \in [0, 1]$ and $\rho_i^{\text{sl}}[t] \in [0, 1]$ split resources between LSS and LTS, leading to $R^{\text{LSS}}[t] = W^{\text{sl}}[t] \cdot R^{\text{total}}[t]$ and $R^{\text{LTS}}[t] = (1 - W^{\text{sl}}[t]) \cdot R^{\text{total}}[t]$, where $(W^{\text{sl}}[t], R^{\text{total}}[t])$

represents $(W_i^{\ell,B,sl}[t], B_i^{\ell}[t])$ for bandwidth or $(\rho_i^{sl}[t], C_i[t])$ and $(\rho_i^{sl}[t], F_i[t])$ for computing resources.

B. Proposed Method

We reformulate $\mathcal{P}$ as a CPOMDP with discount factor γ and, at time t , underlying true state $\mathbf{s}_t$, action $\mathbf{a}_t$, and belief state $\mathbf{b}_t$. The objective is to maximize the expected discounted return of the reward $\mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) = -\sum_{s=0}^{S-1} \sum_{u \in \mathcal{U}} E_u^{\text{Total}}[t_s]$, subject to the expected discounted multiple cost returns. For each sl and platform type $\text{com} \in \{\text{CMP}, \text{CPP}\}$, the costs are

$$\nu_{sl}^{\text{com}}(\mathbf{s}_t, \mathbf{a}_t) = \max_{s \in \mathcal{S}} \max_{i \in \mathcal{I} \cup \{0\}} \max_{u \in \mathcal{U}^{sl}} \left[T_u^{\text{com}}(t_s) - T_u^{\text{M},\text{com}}(t_s) \right]^+, \quad (27)$$

where $[\cdot]^+$ denotes $\max(\cdot, 0)$, and $T_u^{\text{M},\text{com}}(t_s)$ denotes the per-component latency budget, i.e., $\Theta_{iu}^{sl}[t] \cdot T_u^{\text{max}}(t_s)$ for $\text{com} = \text{CMP}$ and $(1 - \Theta_{iu}^{sl}[t]) \cdot T_u^{\text{max}}(t_s)$ for $\text{com} = \text{CPP}$. Using Lagrangian duality with Lagrange multipliers $\{\lambda_{sl,t}^{\text{com}} \geq 0\}$ for all com, sl , and t , we solve the following problem:

$$\min_{\{\lambda_{sl,t}^{\text{com}}\}} \max_{\pi_{\mathcal{O}}} \mathbb{E}_{\pi_{\mathcal{O}}} \left[\sum_{t=0}^{\infty} \gamma^t \left(\mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) - \sum_{\text{sl}, \text{com}} \lambda_{sl,t}^{\text{com}} \nu_{sl}^{\text{com}}(\mathbf{s}_t, \mathbf{a}_t) \right) \middle| \mathbf{b}_t \right]. \quad (28)$$

We then utilize a world model-based agent that employs RSSM to learn model state $\hat{\mathbf{s}}_t = (\mathbf{h}_t, \mathbf{z}_t)$. Here, $\mathbf{h}_t$ is a deterministic state from the RNN encapsulating history, and $\mathbf{z}_t$ is a stochastic state derived from observation embedding vector $\mathbf{e}_t$, output by the graph attention network (GAT) [13], and $\mathbf{h}_t$. Specifically, deterministic state $\mathbf{h}_t$ transitions according to the RNN as $\mathbf{h}_t = f_{\phi}(\mathbf{h}_{t-1}, \mathbf{z}_{t-1}, \mathbf{a}_{t-1})$. This $\hat{\mathbf{s}}_t$ is used to forecast observations $\hat{\mathbf{o}}_t \sim p_{\phi}(\cdot | \hat{\mathbf{s}}_t)$ and rewards and costs $\hat{\mathbf{r}}_t, \hat{\mathbf{c}}_{\text{com}, \text{sl}, i, t} \sim p_{\phi}(\cdot | \hat{\mathbf{s}}_t, \mathbf{a}_t)$. Specifically, to handle dynamic LEO topologies, the world model predicts costs at per-platform granularity. For each platform i , sl, and com, it maintains separate cost prediction heads:

$$\hat{\mathbf{c}}_{\text{com}, \text{sl}, i, t} = \max_{s \in \mathcal{S}} \max_{u \in \mathcal{U}_i^{\text{UL}, \text{sl}}[t]} \left[T_u^{\text{com}}(t_s) - T_u^{\text{M}, \text{com}}(t_s) \right]^+, \quad (29)$$

where the UE set is $\mathcal{U}_i^{\text{UL}, \text{sl}}$ for CMP and $\mathcal{U}_i^{\text{CPP}, \text{sl}}$ for CPP. LEO ($i = 0$) maintains CPP cost heads to account for relay-forwarded computing latency via $\Theta_0^{sl}[t]$. These platform-specific costs are then aggregated to obtain the system-level cost prediction $\hat{\nu}_{sl}^{\text{com}}(\hat{\mathbf{s}}_t, \mathbf{a}_t) = \max_{i \in \mathcal{I} \cup \{0\}} \hat{\mathbf{c}}_{\text{com}, \text{sl}, i, t}$. The world model is trained using a mini-batch $\mathcal{B}$ of trajectories with length T , by minimizing the negative ELBO loss function shown in (30) at the bottom of this page. This objective creates an information bottleneck through the interplay of its two main components. The reconstruction terms in (30) incentivize the model to learn $\hat{\mathbf{s}}_t$ that infers the underlying state of the partially observable environment and encodes

a comprehensive representation at t . Conversely, Kullback-Leibler divergence $\text{KL}(\cdot)$ term regularizes the model, forcing posterior $q_{\phi}(\mathbf{z}_t | \mathbf{h}_t, \mathbf{e}_t)$ to remain close to prior $p_{\phi}(\hat{\mathbf{z}}_t | \mathbf{h}_t)$. This pressure trains p_{ϕ} to capture the true environmental dynamics, which is essential for the subsequent planning phase.

Once the world model is trained, the agent rolls out H steps starting from each state along the trajectories sampled from $\mathcal{B}$, sampling actions from π_{θ} , and predicting the next state using prior p_{ϕ} . Hereafter, t indexes the time step within trajectories sampled from $\mathcal{B}$, and $h \in \{0, \dots, H-1\}$ the imagination step from each t . The actor, reward critic, and multiple cost critics are all trained using these imagined trajectories. The reward critic learns the bootstrapped λ -return, denoted by:

$$R_{t,h}^{\lambda} = \hat{r}_{t,h} + \gamma((1-\lambda)V_{\psi^V}(\hat{\mathbf{s}}_{t,h+1}) + \lambda R_{t,h+1}^{\lambda}), \quad (31)$$

with $R_{t,H}^{\lambda} = V_{\psi^V}(\hat{\mathbf{s}}_{t,H})$. Each cost critic learns a structured cost return $C_{t,h}^{\lambda, \text{sl}, \text{com}}$ for its service-resource pair, replacing rewards with costs. The critics minimize negative log-likelihood losses on their respective returns. The actor is trained based on the Lagrangian advantage function, defined as:

$$A_{t,h}^{\mathcal{L}} = A_{t,h}^V - \sum_{\text{sl}, \text{com}} \lambda_{sl,t}^{\text{com}} \cdot A_{t,h}^{C, \text{sl}, \text{com}}, \quad (32)$$

where $A_{t,h}^V = R_{t,h}^{\lambda} - V_{\psi^V}(\hat{\mathbf{s}}_{t,h})$ and $A_{t,h}^{C, \text{sl}, \text{com}} = C_{t,h}^{\lambda, \text{sl}, \text{com}} - C_{\psi^C}^{\text{sl}, \text{com}}(\hat{\mathbf{s}}_{t,h})$ are the reward and cost advantages, respectively. The actor loss is given as

$$\mathcal{L}(\theta) = -\mathbb{E}_{\pi_{\theta}} \left[\sum_{t=1}^T \sum_{h=0}^{H-1} \gamma^h \left(A_{t,h}^{\mathcal{L}} \cdot \log \pi_{\theta}(\cdot) + \beta_{\mathbf{H}} \mathbf{H}[\pi_{\theta}(\cdot)] \right) \right], \quad (33)$$

where $(\cdot)$ represents $(\mathbf{a}_{t,h} | \hat{\mathbf{s}}_{t,h})$, and $\beta_{\mathbf{H}} \mathbf{H}[\pi_{\theta}(\cdot)]$ is the entropy bonus [14], with coefficient $\beta_{\mathbf{H}}$ updated via $\nabla_{\beta_{\mathbf{H}}} = \mathbb{E}_{\pi_{\theta}} [-(\log \pi_{\theta}(\cdot) + \mathcal{H}_{\text{target}})]$. By compressing $\mathbf{o}_t$ into compact latent states, total inference complexity, which comprises state encoding and action generation, remains comparable to a model-free baseline that processes $\mathbf{o}_t$. Let N_n , N_l , and N_h denote the number of nodes, layers, and hidden dimensions, respectively. The inference complexity of the proposed framework is dominated by the GAT encoder processing $\mathbf{o}_t$, scaling as $\mathcal{O}(N_h^2 \cdot N_n \cdot N_l)$ [13], since the subsequent latent-based action generation is computationally lightweight. In contrast, the actor of the model-free baseline must directly process the high-dimensional $\mathbf{o}_t$ containing features from all N_n nodes. Consequently, the input processing of the baseline actor inherently incurs the same asymptotic complexity of $\mathcal{O}(N_h^2 \cdot N_n \cdot N_l)$ to extract features from the raw input space, resulting in a comparable total inference cost. In practice, world model training can be performed in non-real-time, while only

$$\mathcal{L}(\phi) = \mathbb{E}_{\mathcal{B}} \left[\sum_{t=1}^T \left(-\log p_{\phi}(\hat{\mathbf{o}}_t | \hat{\mathbf{s}}_t) - \log p_{\phi}(\hat{\mathbf{r}}_t | \hat{\mathbf{s}}_t, \mathbf{a}_t) - \sum_{\text{sl}, \text{com}, i} \log p_{\phi}(\hat{\mathbf{c}}_{\text{com}, \text{sl}, i, t} | \hat{\mathbf{s}}_t, \mathbf{a}_t) + \text{KL}[(q_{\phi}(\mathbf{z}_t | \mathbf{h}_t, \mathbf{e}_t)) || p_{\phi}(\hat{\mathbf{z}}_t | \mathbf{h}_t)] \right) \right], \quad (30)$$

Table I
SIMULATION PARAMETERS

Parameter	LSS	LTS	
Number of UEs	20	20	
UE Speed (LSS / LTS)[km/h]	60	15	
Inter-Arrival Time per UE[ms]	H-Exp(20, 200, $\frac{5}{6}$)	H-Exp(40, 600, 0.8036)	
Packet Size[KB]	Uniform(0.75, 1.5)	Uniform(2.25, 4.5)	
Target Latency[ms]	Uniform(40, 60)	Uniform(140, 160)	
Parameter(UE/TBS/LEO)	Value	Parameter(UE/TBS/LEO)	Value
$\Delta T/T_S$ [ms]	100 / 10	$G^{TX/RX}$ [dBi]	12 / 18 / 34.1
NF[dB]	5.0 / 5.0 / 2.0	P_{static}^{TX} [W]	0.65/140/39
P_{cir}^{TX} [mW/MHz]	45 / 75 / 60	P_{static}^{RX} [W]	0.1 / 10 / 5
P_{cir}^{RX} [mW/MHz]	$\frac{2}{3} P_{cir}^{TX}$	P_i^{idle} [W]	5.0
P_i^{gpp} [W]	60.0	ε_C [J/FLOP]	$6.2 \cdot 10^{-12}$
ε_F [J/Byte]	$1.5 \cdot 10^{-11}$	C_i^{max} [FLOP/s]	$19.5 \cdot 10^{12}$
F_i^{max} [GB/s]	768.0	I_{WL} [FLOP/GB]	$1.178 \cdot 10^{15}$
Γ_{WL} [FLOP/bit]	$1.438 \cdot 10^5$	w_{sm}	0.5
w_{mem}	0.5	ρ_{CPP}/σ_{CPP}	0.7 / 0.05
$\rho_{a,LSS}/\rho_{a,LTS}$	0.99 / 0.9	$\sigma_{a,LSS}/\sigma_{a,LTS}$ [packet/s]	0.25 / 0.08
Obs. Dim. $\dim(o_t)$	3,702	Action Dim. $\dim(a_t)$	48
Det. State. Dim. $\dim(h_t)$	256	Sto. State. Dim. $\dim(z_t)$	32

lightweight inference is executed in near-real-time, maintaining real-time compatibility [1]. All $\lambda_{sl}^{com}[t]$ are updated via:

$$\mathcal{L}_\lambda = -\mathbb{E}_{\pi_\theta} \left[\sum_{sl,com} \sum_{t=1}^T \sum_{h=0}^{H-1} \gamma^h \lambda_{sl,t}^{com} \cdot C_{t,h}^{\lambda,sl,com} \right]. \quad (34)$$

The inner policy π_I operates within the resource envelopes set by the outer agent, allocating bandwidth and computing resources at t_s . We employ an equal-share allocation strategy $b_{u,i}^\ell[t_s] = \frac{1}{|\mathcal{U}_i^{\ell,sl}[t]|}$ for all $t, s, \ell, i \in \mathcal{I} \cup \{0\}$, and $u \in \mathcal{U}_i^{\ell,sl}[t]$, and $f_{u,i}^{CPP}[t_s] = \frac{1}{|\mathcal{U}_i^{CPP,sl}[t]|}$ for all $t, s, i \in \mathcal{I}$, and $u \in \mathcal{U}_i^{CPP,sl}[t]$, where computational complexities are asymptotically linear in the size of each UE set.

IV. NUMERICAL RESULTS

To evaluate the stochastic dynamics learning, we set a representative SAMEC scenario with four TBSs and 40 UEs in a 2×2 km² area. A LEO satellite operates at an altitude of 600 km, traveling at 7.8 km/s, and maintains continuous connectivity through next-hop routing when the elevation angle reaches 30° [15]. All links operate in the C-band with 10 MHz bandwidth, and the noise spectral density is N_0 [dBW/Hz] = $-174 + NF$, where NF denotes the noise figure.

The channel models follow 3GPP TR 38.901 [16] for terrestrial links and TR 38.811 [15] incorporating atmospheric attenuation per ITU-R P.618 [17] for NTN links, where parameters such as the Rician K-factor are determined by dynamic geometry. We model time-correlated stochastic processes using first-order auto-regressive (AR(1)) models represented as $z_t = \rho z_{t-1} + \epsilon_t$, where z_t denotes the stochastic variable of interest and $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$. The packet inter-arrival times per UE follow a correlated hyper-exponential distribution H-Exp($\mu_1, \mu_2, \bar{p}$), which is a mixture of two exponential

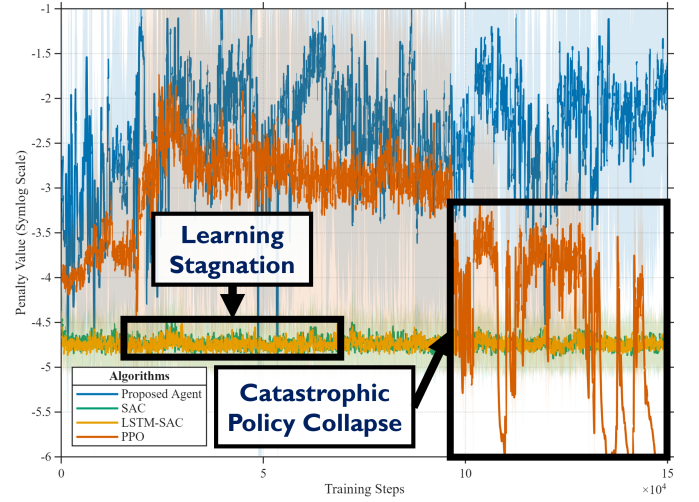


Figure 3. Training curves of penalty function values. The y-axis uses a symmetric logarithmic scale to accommodate the wide performance range.

distributions with means μ_1 and μ_2 and a mixing probability $\bar{p}$, with packet sizes varying via AR(1) parameterized by $(\rho_{a,sl}, \sigma_{a,sl})$ for each sl. $\eta_{CPP}[t]$ varies over time via AR(1) with parameters $(\rho_{CPP}, \sigma_{CPP})$. Due to space limitations, detailed modeling of other aspects, i.e., wireless channel loss, which includes NTN loss, line-of-sight (LoS) transitions, shadowing, and coherence time mechanics, is omitted but strictly follows the specifications in the referenced standards. UE mobility is modeled as a random walk with boundary reflections. All simulation parameters are detailed in Table I.

We utilize proximal policy optimization (PPO) as an on-policy, soft actor-critic (SAC) as an off-policy, and long short-term memory (LSTM)-SAC as an RNN-DRL baseline, respectively. All agents use $B = 32$, with all learning rates set to 10^{-4} , 5-layer heads, and N_h set to 512. LSTM-SAC and the proposed agent use $T = 64$ for temporal modeling, while the proposed agent employs $H = 15$. Fig. 3 compares penalty function values during training. For fair comparison, we use a fixed penalty coefficient of 10.0 for all agents, instead of their individually updated Lagrangian multipliers $\lambda_{sl,t}^{com}$. As the reward maximizes negative energy consumption, the penalty convergence implies joint energy-latency optimization. PPO shows quick initial convergence but exhibits instability after 10k steps, achieving substantially worse performance than the proposed method. SAC and LSTM-SAC demonstrate the poorest performance, indicating challenges in handling partially observable environments with deterministic states. Fig. 4 illustrates latency deviations between actual and target latencies across algorithms for each sl across CMP and CPP. The proposed agent consistently achieves the smallest deviation with compact distributions, while the baseline algorithms exhibit significantly larger variances and outliers. For communication services, the proposed agent maintains mean deviations within 20 ms for both LSS and LTS, staying close to the targets. In contrast, SAC and LSTM-SAC show deviations exceeding 150 ms and 280 ms, respectively, while PPO

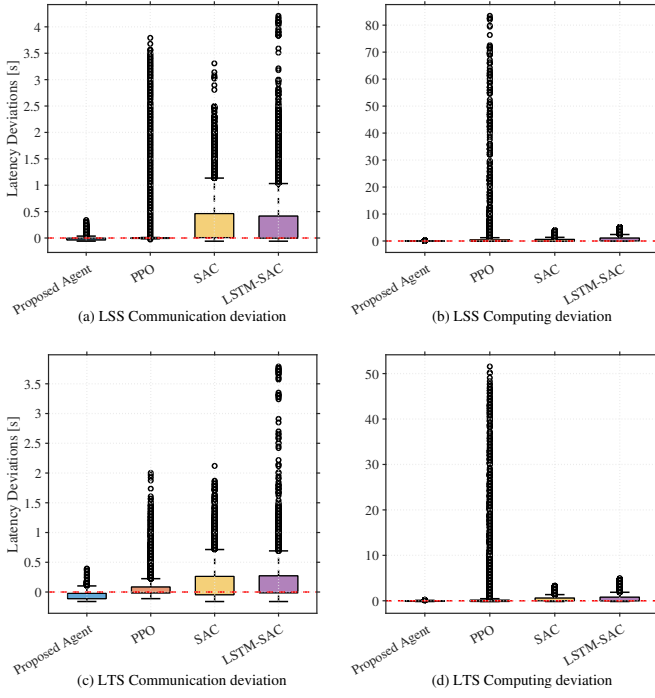


Figure 4. Comparison of latency constraint deviations among agents.

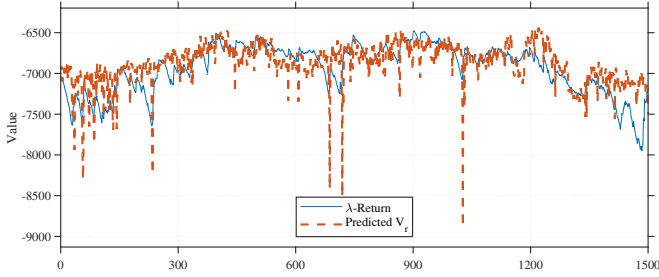


Figure 5. Comparison between predicted and actual reward returns, where V_T represents $V_{\psi,V}(\mathbf{s}_t)$ and the λ -return corresponds to $R_{t,0}^\lambda$.

frequently overshoots targets. A similar trend is observed in the computation services, where the proposed agent maintains deviations below 65 ms, whereas LSTM-SAC exhibits extreme cases exceeding 4 s. These results confirm that the proposed algorithm effectively minimizes latency deviations with a low mean error and reduced variance, demonstrating superior adaptability across dynamic environments. To understand why the proposed agent outperforms the baselines, Fig. 5 compares the multiple step predicted and actual reward returns. The proposed agent estimates returns and values up to H . The mean absolute percentage error (MAPE) between predictions and actual values is 2.55%, demonstrating that the critic effectively estimates values using s_t . This explains the superior performance observed in Fig. 3 and Fig. 4. These evaluations demonstrate that the proposed model-based DRL consistently outperforms conventional baselines in both training efficiency and constraint satisfaction.

V. CONCLUSION

This paper is the first to demonstrate that explicit stochastic dynamics modeling via world models is critical for the SAMEC orchestration problem under partial observability. By enabling planning-based imagination for proactive constraint satisfaction rather than reactive penalties, our approach achieves 2.55% MAPE in value prediction and limits QoS violations below 20 ms, versus 150–280 ms with model-free baselines. Future work should evaluate the fidelity of learned latent representations in stochastic dynamics and explore alternative inner policies, such as proportional-fair and max-min, to identify optimal policy combinations. Furthermore, the scope can be extended to investigate its scalability in dense network scenarios while accounting for inter-cell interference.

REFERENCES

- [1] O-RAN-WG1-CCIN-TR, *Communication and Computing Integrated Networks*, O-RAN Alliance, Jul. 2024, vol. 01.00, rel. 004.
- [2] M. Chen and Y. Hao, "Task offloading for mobile edge computing in software defined ultra-dense network," *IEEE Journal on Selected Areas in Communications*, vol. 36, no. 3, pp. 587–597, 2018.
- [3] T. Kim, J. Kwak, and J. P. Choi, "Satellite edge computing architecture and network slice scheduling for IoT support," *IEEE Internet of Things journal*, vol. 9, no. 16, pp. 14938–14951, 2021.
- [4] I. Choi, S. Kim, Y. Lee, and J. P. Choi, "A scalable multicontroller SDN framework for LEO mega-constellation via topology virtualization," *IEEE Internet of Things Journal*, vol. 12, no. 16, pp. 33 311–33 327, 2025.
- [5] C. Ding, J.-B. Wang, M. Cheng, M. Lin, and J. Cheng, "Dynamic transmission and computation resource optimization for dense LEO satellite assisted mobile-edge computing," *IEEE Transactions on Communications*, vol. 71, no. 5, pp. 3087–3102, 2023.
- [6] N. Cheng, F. Lyu, W. Quan, C. Zhou, H. He, W. Shi, and X. Shen, "Space/aerial-assisted computing offloading for IoT applications: A learning-based approach," *IEEE Journal on Selected Areas in communications*, vol. 37, no. 5, pp. 1117–1129, 2019.
- [7] G. Chen, S. Qi, F. Shen, Q. Zeng, and Y.-D. Zhang, "Information-aware driven dynamic LEO-RAN slicing algorithm joint with communication, computing, and caching," *IEEE Journal on Selected Areas in Communications*, vol. 42, no. 5, pp. 1044–1062, 2024.
- [8] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv preprint arXiv:1312.6114*, 2013.
- [9] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson, "Learning latent dynamics for planning from pixels," in *International Conference on Machine Learning*. PMLR, 2019, pp. 2555–2565.
- [10] S. Williams, A. Waterman, and D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [11] H. Holtkamp, G. Auer, V. Giannini, and H. Haas, "A parameterized base station power model," *IEEE Communications Letters*, vol. 17, no. 11, pp. 2033–2035, 2013.
- [12] J. W. Choi, D. Bedard, R. Fowler, and R. Vuduc, "A roofline model of energy," in *2013 IEEE 27th International Symposium on Parallel and Distributed Processing*. IEEE, 2013, pp. 661–672.
- [13] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," *arXiv preprint arXiv:1710.10903*, 2017.
- [14] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International Conference on Machine Learning*. PMLR, 2018, pp. 1861–1870.
- [15] 3rd Generation Partnership Project (3GPP), "TR 38.811 : Study on NR to support non-terrestrial networks (Release 15)."
- [16] —, "TR 38.901 : Study on channel model for frequencies from 0.5 to 100 GHz (Release 19)."
- [17] P. Series, "Propagation data and prediction methods required for the design of earth-space telecommunication systems," *Recommendation ITU-R*, pp. 618–12, 2015.